

[Slingin' Macros]

Technical Design Documentation

Technical Requirements

Engine: Unity **2022.3.36f1**

Version Control: **Git**

Itch Page: <https://itch.io/profile/slingin-macros>

Enemy Behavior

Enemies chose a lane to walk down. Enemies use a state machine to control their mood. Enemies play different animations depending which mood they are in. The mood states are as follows: calm, frustrated, angry, enraged(if they reach the bar), and satisfied (if fed before reaching the bar). Enemies reaching the bar reduces the Player's health (tip jar fullness).

Enemy Spawning / De-Spawning

Enemies spawn at a randomly selected spawn point. Enemies enter the level and walk to a randomly picked lane, then they walk towards the bar where the player is. Whenever an enemy has been fed, or has reached the bar to complain, the enemy will be despawned. An object pool will be used to recycle Enemies.

Player/Enemy Health

Both the Player and Enemies have health-like systems. The Player's health is represented by a "tip jar" that is projected to be full at the end of the night. When the Player "takes damage" this projection reduces, this indicates to the Player that damage was taken. The Player "takes damage" when an Enemy travels close enough to the bar to trigger an event. Enemies become satisfied when the Player successfully hits them with a food projectile, satisfied Enemies despawn.

Food Throwing

Food can be grabbed and thrown from the sockets on the bar. These food throwables will collide with the customers via trigger collider. Once collision is detected the customer will call it's spawner's despawn method and the food will call the gameObject destroy method. If no collision is detected in the scene after a certain amount of time ,default 5 seconds, the food will call the gameObject destroy method.

Sound Manager

Uses a sound manager to scale sounds played in the game based on settings configured by the Player in the main menu.

User Interface Manager

A UI Manager is used to distribute data from the scene to UI elements.

Input Management

Input will be handled with an HVRPlayerInput Component which wraps Hurricane VR's custom system. This component can be attached to the Player, then tweaked, to give VR control to a User. This component will invoke an event when a corresponding key is pressed, these input events call custom player actions unrelated to VR motion.

VR movement will be handled in the HVRPlayerInput Component wrapper for Hurricane VR's custom input system. Only custom player actions will have events relating to them.